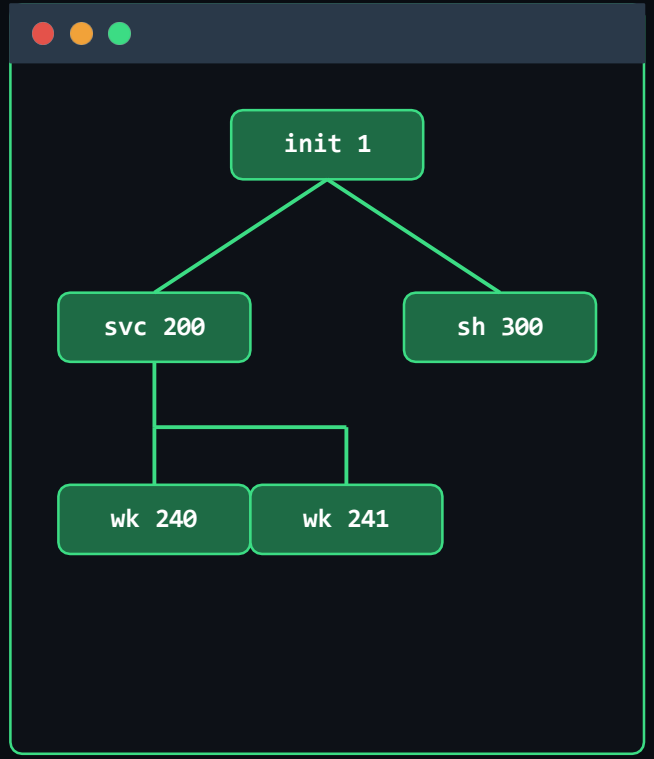
Linux Process Tree Visualizer

An Interactive C++ Tool for Understanding Linux Process Hierarchy

Batch No: 4

1. Amrutha Varshini – 2520080066
2. Hasith Varma – 2520090125
3. Keerthana – 2520090132



$ problem

Problem Statement

Many processes at once

Linux runs dozens or hundreds of processes together.

Hierarchy is hard to see

Beginners find it hard to know which process created another.

Flat lists

Basic listings do not always make parent-child links obvious.

Manual effort

Tracing parent IDs by hand takes time.

PROBLEM WE AIM TO SOLVE

Give learners a simple, read-only terminal tool that rebuilds the parent-child process hierarchy from real Linux data and shows it as a readable tree.

Objectives and Proposed Solution

1. Collect real process information
2. Rebuild parent-child relationships
3. Display a readable process tree
4. Search and inspect processes
5. Show basic process and memory statistics

BEFORE: flat list

```
PID   PPID  NAME
1     0     system
200   1     service
240   200   worker
300   1     shell
```

▼

AFTER: process tree

```
system [PID 1]
├── service [PID 200]
│   └── worker [PID 240]
└── shell [PID 300]
```

$ comparison

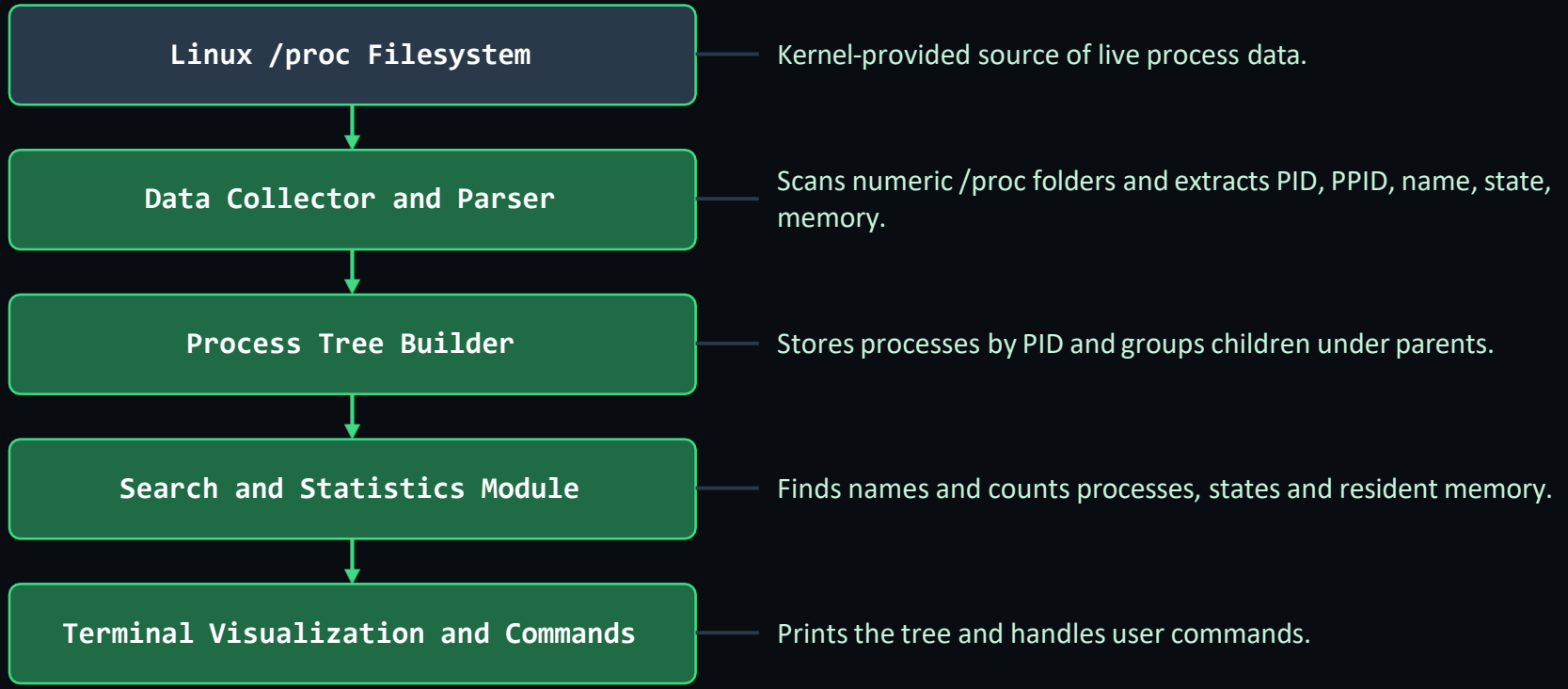
Existing vs Proposed System

Aspect	Existing approaches	Our proposed system
Hierarchy	Individual process listings	Hierarchical process tree
Parent link	Manual inspection of PPIDs	PID and PPID shown together
Finding a process	Repeated terminal commands	Process-name search (case-insensitive)
Details / summary	Several separate commands	info and stats commands in one tool
Updating	Re-run commands	On-demand refresh command

Note: existing Linux tools are powerful and some can show trees. Our goal is a focused, beginner-friendly learning tool.

$ architecture

System Architecture



$ technologies

Technologies Used

C++17
Core programming language.

Linux / Ubuntu on WSL
Execution environment.

/proc filesystem
Source of process information.

GNU g++
Compiler.

Make
Build automation.

C++ standard library
Filesystem access, maps, vectors, strings, I/O.

Develop and run in Ubuntu on Windows using WSL

Implementation and Working

1. Scan numeric folders in /proc
2. Read each process stat file
3. Extract PID, PPID, name, state, memory
4. Store processes in a map by PID
5. Group each child under its parent
6. Traverse the hierarchy recursively
7. Print the tree in the terminal
8. Refresh the snapshot when asked

```
for each numeric dir in /proc:
  read stat -> pid, ppid,
      name, state, memory
  procs[pid] = info
for each p in procs:
  children[p.ppid].push(p.pid)
print(pid, depth):
  show name; for c in children[pid]:
      print(c, depth + 1)
```

```
system [PID 1]
├── service [PID 200]
│   └── worker [PID 240]
└── shell [PID 300]
```

Illustrative

$ concepts

Data Structures and OS Concepts

Map
Finds a process by its PID.

Vector
Stores the children of a parent.

Tree
Shows parent-child relations.

Recursion
Prints nested branches.

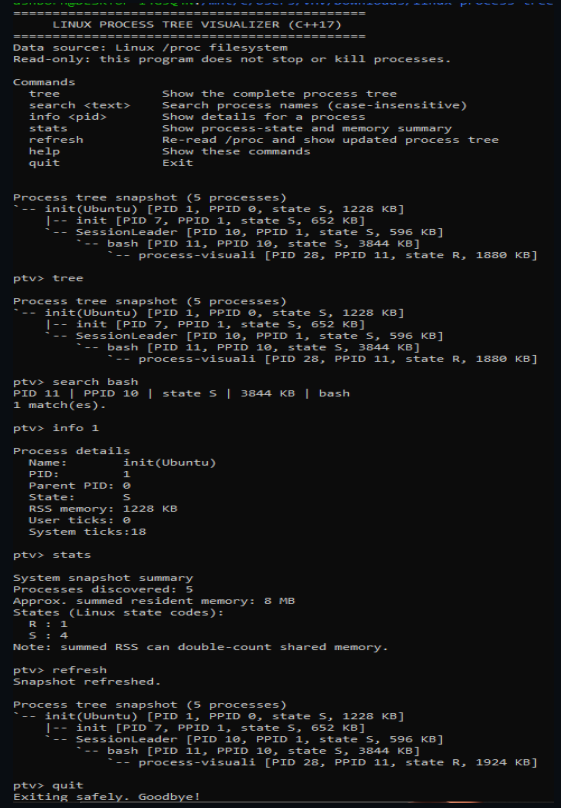
Linux /proc
Gives live process information.

PID / PPID
Process ID and its parent ID.

Process states
Show what a process is doing now.

Why it fits DSA + OS: a process hierarchy is a real tree, built with a map and vectors, printed with recursion, from real operating system data.

Demonstration and Features



```
==============================================
      LINUX PROCESS TREE VISUALIZER (C++17)
==============================================
Data source: Linux /proc filesystem
Read-only: this program does not stop or kill processes.

Commands
  tree              Show the complete process tree
  search <text>     Search process names (case-insensitive)
  info <pid>        Show details for a process
  stats             Show process-state and memory summary
  refresh           Re-read /proc and show updated process tree
  help              Show these commands
  quit              Exit


Process tree snapshot (5 processes)
`-- init(Ubuntu) [PID 1, PPID 0, state S, 1228 KB]
    |-- init [PID 7, PPID 1, state S, 652 KB]
    `-- SessionLeader [PID 10, PPID 1, state S, 596 KB]
        `-- bash [PID 11, PPID 10, state S, 3844 KB]
            `-- process-visuali [PID 28, PPID 11, state R, 1880 KB]

ptv> tree

Process tree snapshot (5 processes)
`-- init(Ubuntu) [PID 1, PPID 0, state S, 1228 KB]
    |-- init [PID 7, PPID 1, state S, 652 KB]
    `-- SessionLeader [PID 10, PPID 1, state S, 596 KB]
        `-- bash [PID 11, PPID 10, state S, 3844 KB]
            `-- process-visuali [PID 28, PPID 11, state R, 1880 KB]

ptv> search bash
PID 11 | PPID 10 | state S | 3844 KB | bash
1 match(es).

ptv> info 1

Process details
  Name:       init(Ubuntu)
  PID:        1
  Parent PID: 0
  State:      S
  RSS memory: 1228 KB
  User ticks: 0
  System ticks:18

ptv> stats

System snapshot summary
Processes discovered: 5
Approx. summed resident memory: 8 MB
States (Linux state codes):
  R : 1
  S : 4
Note: summed RSS can double-count shared memory.

ptv> refresh
Snapshot refreshed.

Process tree snapshot (5 processes)
`-- init(Ubuntu) [PID 1, PPID 0, state S, 1228 KB]
    |-- init [PID 7, PPID 1, state S, 652 KB]
    `-- SessionLeader [PID 10, PPID 1, state S, 596 KB]
        `-- bash [PID 11, PPID 10, state S, 3844 KB]
            `-- process-visuali [PID 28, PPID 11, state R, 1924 KB]

ptv> quit
Exiting safely. Goodbye!
```

`tree`	Show the process tree
`search bash`	Find names (any case)
`info 1`	Details of PID 1
`stats`	Counts and memory total
`refresh`	Take a new snapshot
`quit`	Exit the program

Read-only: never kills or modifies processes.

Testing and Validation

Test case	Expected result
Application startup	Program starts and shows prompt
Process tree display	Indented tree with PID and PPID
Process-name search	Matching names shown, any case
Existing PID details	Details of that process shown
Invalid PID input	Clear message, no crash
Statistics display	Counts, states and memory total
Refresh after a process starts/exits	Snapshot reflects the change
Graceful exit	Program quits cleanly

$ value

Innovation, Applications and Future Scope

IMPLEMENTED VALUE

- Tree, search, details and stats in one tool
- Uses real Linux process data
- Safe, read-only learning environment

POTENTIAL USERS

- Operating Systems students
- Developers learning Linux process management
- System administrators inspecting relationships

FUTURE (PROPOSED)

- Automatic periodic refresh
- CPU % across snapshots
- Creation and termination history
- Highlight resource-heavy processes
- Graphical interface

Conclusion and Q&A

- Problem: hierarchy is hard for beginners
- Tech: C++17, /proc, g++, Make
- Method: map + vectors + recursive tree
- Value: read-only learning tool
- Next: auto refresh, CPU %, history, GUI

GitHub: https://github.com/amrutha-0123/KLH-CSIT-AIDS-2026-27-Team4-FDFM

Thank You